Final Project Summary: Player Analytics for an Online Game

Group: E & D Group | Group Members: Ethan Oxley, Daniel Warren

Arizona State University

IFT 530: Advanced Database Management Systems

Ashish Gulati

October 27, 2026

Final Project Summary: Player Analytics for an Online Game

Project Summary

We are building the database behind a live, free-to-play online game. Not the game itself — the system the business runs it from. A game like this earns nothing the moment somebody installs it. It earns weeks later, from the small share of players who decide it is worth paying for, and it only reaches those players by paying for their attention first. So the questions worth asking cut across areas that normally sit in separate systems: who the player is, what they did, what they paid, and whether we should trust the payment.

The database is called E_D_Group, after our group. A SQL Server name cannot hold spaces or an ampersand, so E & D Group becomes E_D_Group. We designed it for a hundred thousand players and for several systems feeding it at once — a game client, a mobile SDK, a billing API and two app stores all write into the same tables, and every fact row records which one sent it. We went with a star schema because that is what reporting tools expect to find, and because it keeps the busiest table narrow.

Six dimensions describe the player from different angles, and that is the whole point. We can slice a player by age band, by the device they play on, by where they signed up, by which campaign brought them in and what that cost, by how long they have been around, and by where they sit in the lifecycle — new, active, at risk, churned or banned. Their account slices again by plan, by billing status and by whether it renews on its own. Those cuts stack. The question is rarely how many players churned. It is which plan, on which device, from which campaign, in which region.

Three fact tables record what happened. The first is a single stream of player events covering both session boundaries and in-game actions, which is where daily and monthly active

players come from and where we measure level difficulty. The second is payments: store sales, subscription charges and refunds. The third is the subscription lifecycle, which we kept separate because a downgrade and a cancellation move no money at all. They would have nowhere to live otherwise, and they are exactly the events that tell us what revenue is about to do.

What the Database Is Useful For

Storing the data is only half the job. Each of these tables exists, at the grain it does, because it answers a question somebody running this game has to answer every week. Right now those answers are scattered across a game server, a billing provider and an ad dashboard that do not talk to each other. Pulling them into one schema turns a reporting request into a single query. Below are the questions we designed backwards from, and where each answer comes from.

Table 1
The questions we built the database to answer, and where each answer comes from.

What we need to know	Where the answer comes from
How many people played today, and is that going up or down?	Session rows in FactUserEvent rolled up by day in vwDailyActiveUsers, with a rolling 30-day count beside each day so we see the trend and not just today's number.
Do paying players actually play more, or do they only pay?	FactUserEvent joined through DimAccount to DimPlan in vwFreeVsPaidValue, which splits sessions, playtime and spend by whether the plan is a paid one.
Which campaigns paid for themselves?	DimCampaign, with the cost of acquiring each player carried on DimUser, set against what those same players spent in FactPayment.
Where in the game do players quit?	LevelNumber and Outcome on FactUserEvent, walked level by level by curLevelDifficulty, which flags any level players lose more often than they win.
Should we trust this payment?	The location on FactPayment against where the player was last seen and how long ago, in vwSuspiciousTransactions. fnGeoDistanceKm turns the two sets of coordinates on DimGeography into kilometers, so an impossible trip is something we measure instead of guess at.
What is recurring revenue doing?	MRRDeltaUSD on FactSubscription, with PrevPlanID telling an upgrade apart from a downgrade, so a cancel and a downgrade still show up even though neither one moves money.
What is a player worth in a given month?	spMonthlyActiveUsers hands back active players, new against returning, revenue after refunds, and average revenue per player for whatever month we ask for.
Who changed a plan's price, and when?	DimPlanAudit, written only by the trigger on DimPlan. It keeps the old and new values, who made the change, and the time it happened.

Geography and Suspicious Transactions

Geography is one dimension we point at five times, and that is what lets us do fraud reporting without a second system. A player has a registered location from signup, and a last known location with a timestamp that updates as they play. Their account has a billing location. Every event and every payment records where it came from. All five point at the same table, so we can compare any two of them directly.

That turns a vague worry into a query we can actually run. A payment from a country the player never registered in is worth a look. A payment from somewhere the dimension marks high risk is worth more of one. And because the geography table carries coordinates, we can measure the distance between where somebody was last seen and where their money just moved, against the time between the two. A session in Phoenix and a payment from Lagos twenty minutes later is not a trip anybody took. Each of those is just a condition on a join between tables we already have, which is the argument for putting location in a dimension rather than repeating a country code on every row.

Designing for Scale

At a hundred thousand players the event table is the one that decides whether any of this works, so we kept it to integer keys, one timestamp and a few short coded columns. Everything descriptive lives in a dimension and is reached by a join. Session membership is a plain integer on the event rather than a table of its own, because a session has nothing to say beyond which events belong to it, and its length is just the span of those events.

Every table uses a surrogate integer key instead of a natural one, so a player changing their username or a plan getting renamed does not ripple through the fact tables. Fact table keys are IDENTITY columns, because at this volume nobody should be handing out keys by hand.

Past the primary and foreign keys, we index the two timestamp columns every time-series query filters on, plus the foreign keys the fact tables join through. Check constraints hold the coded columns to a fixed set of values, which matters more the more systems are writing to the table.

Database Design

Dimensions hold what is true about a player, an account, a plan, a product, a campaign or a place. Facts hold what happened, each at a grain we state up front — one player action, one payment, one subscription event. Being clear about the grain is what keeps a query from double-counting revenue or inflating an active player count.

We create tables parent-first, so every foreign key points at something that already exists: the four tables with no foreign keys of their own, then DimUser, then DimAccount, then the three fact tables. Nothing references in a circle. DimGeography, DimPlan, DimProduct and DimCampaign have no outbound foreign keys at all, so nothing closes a loop and we can drop the tables in the reverse of the order we made them.

The schema is ten tables, which is the cap the assignment sets and also what this design needs: six dimensions, three transactional tables and one audit table. An earlier draft had a marketing click table too. We cut it and moved the acquiring campaign and its cost onto DimUser, so we can still work out cost per acquisition without an eleventh table. The assignment asks for one or two fact tables and we have three, because the three grains really are different — one player action, one payment, one subscription event — and squashing them together would force nullable columns that mean nothing for two rows out of every three.

Table 2
The ten tables, what each one is for, and what it holds.

Table	Role	Rows	What it holds
DimGeography	dimension	10	Every place the system knows about, with its region, its coordinates and how risky it is. Five columns across four tables point at this one table, which is what lets registered location, last known location, billing location and transaction location all be compared against each other.
DimUser	dimension	10	One row per person. This is the table customer analysis slices on: age band, device, lifecycle stage, signup cohort, acquiring campaign, acquisition cost, and both of the geography columns.
DimAccount	dimension	10	One row per billing entity. Separate from the person because a person can close an account and open another, and because plan, status and billing address belong to the account rather than to them.
DimPlan	lookup	10	The subscription plans an account can be on. This is the lookup table the audit requirement is built on, because price and perks are exactly the kind of reference data that changes and that somebody later needs to account for.
DimProduct	dimension	10	The store catalog behind one-time sales, kept separate so a price change does not rewrite purchase history.
DimCampaign	dimension	10	Every campaign the game has run, with the channel it ran on and what it was budgeted. Without the budget there is no way to say whether a channel paid for itself.
FactUserEvent	fact	48	One row per action, session boundaries included. The finest grain in the database and the single busiest table, which is why it carries nothing but keys, a timestamp and a handful of short codes.
FactPayment	fact	30	One row per payment: store sales, subscription charges and refunds, each tagged with where the transaction came from.
FactSubscription	fact	23	One row per subscription event. Separate from payments because the events that matter most to retention, a downgrade and a cancel, move no money at all and would have nowhere to live otherwise.
DimPlanAudit	audit	grows	The change history of DimPlan. Written only by a trigger, never by hand.

A Note on Row Counts

The assignment asks for ten rows in each dimension table and 20 to 50 in each transactional table, and that is exactly what we load. All six dimensions carry ten rows. FactUserEvent holds 48, FactPayment 30 and FactSubscription 23. DimPlanAudit is not loaded at all — the trigger writes it, so it starts empty and fills up when the audit test runs.

Small data makes a reporting database hard to show off. Ten players spread thin across a month give you a flat line and a lot of empty result sets. So we laid the data out by hand instead of generating it at random. The window is 30 days and crosses a month boundary, so the monthly procedure has more than one row to return. The twelve sessions sit on nine days, and a few of those days carry two or three different players, so daily active users moves between one and

three instead of sitting on one. Nine of the ten accounts are open and they cover both a free and a paid plan, which is what the free-versus-paid view needs in order to compare anything. Level five is lost three times out of four, which is the churn risk the cursor reports. And three payments sit in countries the account holder was nowhere near minutes earlier, which is what the suspicious transaction view picks up.

None of that is padding. Every one of those is something a view, the procedure or the cursor depends on, and at ten players a random draw can easily miss all of them. Our generator checks each one before it writes a single INSERT, so a change that would quietly empty a result set breaks at build time instead of in front of whoever is reading the report. The schema is built for a hundred thousand players; the row counts are what the assignment asks a submitted script to carry.

Table Definitions

Here is every table, column by column: the data type and size, whether a value is required, what constrains it, whether it is part of the primary key, and why we have it. Constraint names follow the pattern we use through this course — PK_ for primary keys, FK_ for foreign keys, UQ_ for unique constraints, CK_ for check constraints and DF for defaults.

DimGeography

Every place the system knows about, with its region, its coordinates and how risky it is. Five columns across four tables point at this one table, which is what lets registered location, last known location, billing location and transaction location all be compared against each other.

Table 3
DimGeography — column specification.

Column Name	Data Type	Size	Null / NOT Null	Constraint	PK	Purpose of the column
GeoID	INT	4	NOT NULL	PK_Geography	Yes	Surrogate key. Five different columns point at this table, so one row answers every question about place.
CountryCode	CHAR	2	NOT NULL			ISO country code. Fixed-width CHAR because it is always two letters.
CountryName	VARCHAR	60	NOT NULL			Country name for reports.
Region	VARCHAR	40	NOT NULL	CK_Geo_Region		North America, EMEA, APAC or LATAM. The coarse grouping most executive reporting rolls up to.
City	VARCHAR	60	NOT NULL			City, so location is finer than a country when it needs to be.
Latitude	DECIMAL	9,6	NOT NULL	CK_Geo_Latitude		Between -90 and 90. Feeds the distance calculation behind the impossible-travel check.
Longitude	DECIMAL	9,6	NOT NULL	CK_Geo_Longitude		Between -180 and 180. The other half of that calculation.
RiskTier	VARCHAR	10	NOT NULL	CK_Geo_RiskTier		Low, Medium or High. Lets a payment be screened on where it came from without hard-coding a country list into a query.

DimUser

One row per person. This is the table customer analysis slices on: age band, device, lifecycle stage, signup cohort, acquiring campaign, acquisition cost, and both of the geography columns.

Table 4
DimUser — column specification.

Column Name	Data Type	Size	Null / NOT Null	Constraint	PK	Purpose of the column
UserID	INT	4	NOT NULL	PK_User	Yes	Surrogate key for the person.
Username	VARCHAR	30	NOT NULL	UQ_User_Username		Public display name. Unique across the game.
Email	VARCHAR	100	NOT NULL			Login and contact address.
SignupDate	DATE	3	NOT NULL			When the person first registered. The anchor every cohort and retention measure is built on.
RegisteredGeoID	INT	4	NOT NULL	FK_User_RegGeo		Where the account was registered. The first half of the geography mismatch test.
LastSeenGeoID	INT	4	NULL	FK_User_LastGeo		Last known location. NULL until the first session. Compared against the registered location and against where a payment came from.

Column Name	Data Type	Size	Null / NOT Null	Constraint	PK	Purpose of the column
LastSeenAt	DATETIME2	8	NULL			When they were last seen in that location. With the two geography columns, this is what makes impossible travel detectable.
AgeBand	VARCHAR	10	NOT NULL	CK_User_AgeBand		13-17, 18-24, 25-34, 35-49 or 50+. Banded rather than a birth date, which keeps the segment useful without storing a precise identifier.
PreferredDevice	VARCHAR	20	NOT NULL	CK_User_Device		iOS, Android, PC or Console. The platform they mostly play on.
LifecycleStage	VARCHAR	12	NOT NULL	CK_User_Lifecycle		New, Active, AtRisk, Churned or Banned. The single column most segmentation starts from.
AcqCampaignID	INT	4	NULL	FK_User_Campaign		The campaign that acquired this person. NULL means organic, which is why it is optional.
AcqCostUSD	DECIMAL	8,2	NULL	CK_User_AcqCost		What acquiring them cost. Carrying it here means cost per acquisition is computable without a click-level table.

DimAccount

One row per billing entity. Separate from the person because a person can close an account and open another, and because plan, status and billing address belong to the account rather than to them.

Table 5
DimAccount — column specification.

Column Name	Data Type	Size	Null / NOT Null	Constraint	PK	Purpose of the column
AccountID	INT	4	NOT NULL	PK_Account	Yes	Surrogate key for the billing entity.
UserID	INT	4	NOT NULL	FK_Account_User		The person who holds the account. One person can hold several over time, which is why these are separate tables.
AccountNumber	VARCHAR	20	NOT NULL	UQ_Account_Number		The external identifier billing systems and support tickets quote. Unique.
PlanID	INT	4	NOT NULL	FK_Account_Plan		The plan the account is on right now. Mandatory: every account sits on some plan, even the free one.
AccountStatus	VARCHAR	12	NOT NULL	CK_Account_Status		Trial, Active, PastDue, Cancelled or Suspended. PastDue and Suspended are the states that matter to revenue and to fraud.
OpenedDate	DATE	3	NOT NULL			When the account was opened.
ClosedDate	DATE	3	NULL	CK_Account_Dates		When it closed. NULL means still open. Must not precede OpenedDate.
IsAutoRenew	BIT	1	NOT NULL	DF 1		Whether the plan renews on its own. A leading indicator of churn before a cancellation ever lands.

Column Name	Data Type	Size	Null / NOT Null	Constraint	PK	Purpose of the column
BillingGeoID	INT	4	NOT NULL	FK_Account_BillGeo		Where the billing address sits. A third place a location can differ from the other two.

DimPlan

The subscription plans an account can be on. This is the lookup table the audit requirement is built on, because price and perks are exactly the kind of reference data that changes and that somebody later needs to account for.

Table 6
DimPlan — column specification.

Column Name	Data Type	Size	Null / NOT Null	Constraint	PK	Purpose of the column
PlanID	INT	4	NOT NULL	PK_Plan	Yes	Surrogate key for the plan.
PlanName	VARCHAR	30	NOT NULL	UQ_Plan_Name		Free, Plus, Premium and so on. Unique so two plans cannot share a label.
PriceUSD	DECIMAL	6,2	NOT NULL	CK_Plan_Price		Recurring price, 0.00 on the free plan. Exact decimal because it is money. Must be >= 0.
BillingPeriod	VARCHAR	10	NOT NULL	CK_Plan_Period		Monthly, Annual or None. Needed to turn a price into monthly recurring revenue.
IsPaid	BIT	1	NOT NULL	DF 0		1 when the plan costs money. This is the column the free-versus-paid analysis splits on.
TrialDays	INT	4	NOT NULL	CK_Plan_Trial		Length of the free trial, 0 when there is none. Must be >= 0.
AdsEnabled	BIT	1	NOT NULL	DF 1		Whether ads are shown. Explains ad-watch event volume by plan.
EffectiveDate	DATE	3	NOT NULL			When this plan and price took effect.

DimProduct

The store catalog behind one-time sales, kept separate so a price change does not rewrite purchase history.

Table 7
DimProduct — column specification.

Column Name	Data Type	Size	Null / NOT Null	Constraint	PK	Purpose of the column
ProductID	INT	4	NOT NULL	PK_Product	Yes	Surrogate key for the item.

Column Name	Data Type	Size	Null / NOT Null	Constraint	PK	Purpose of the column
ProductName	VARCHAR	60	NOT NULL			Item name in the store.
ProductCategory	VARCHAR	20	NOT NULL	CK_Product_Category		Currency, Cosmetic, BattlePass or Subscription. Lets revenue be split by what people actually buy.
ListPriceUSD	DECIMAL	6,2	NOT NULL	CK_Product_Price		List price. Must be > 0; a free item is not a store product. Kept here so a price change does not rewrite sales history.
IsActive	BIT	1	NOT NULL	DF 1		Whether the item is still for sale. Retired items keep their history.
ReleaseDate	DATE	3	NOT NULL			When the item went on sale.

DimCampaign

Every campaign the game has run, with the channel it ran on and what it was budgeted. Without the budget there is no way to say whether a channel paid for itself.

Table 8
DimCampaign — column specification.

Column Name	Data Type	Size	Null / NOT Null	Constraint	PK	Purpose of the column
CampaignID	INT	4	NOT NULL	PK_Campaign	Yes	Surrogate key for the campaign.
CampaignName	VARCHAR	60	NOT NULL			Campaign name as the marketing team refers to it.
Channel	VARCHAR	30	NOT NULL	CK_Campaign_Channel		Paid Search, Social, Influencer, Email or Organic. Restricted so a typo cannot create a channel.
CostModel	VARCHAR	10	NOT NULL	CK_Campaign_CostModel		CPC, CPM or CPI. How the channel bills.
StartDate	DATE	3	NOT NULL			First day the campaign ran.
EndDate	DATE	3	NULL	CK_Campaign_Dates		Last day it ran. NULL means it is still running. Must not precede StartDate.
BudgetUSD	DECIMAL	10,2	NOT NULL	CK_Campaign_Budget		Total committed spend. The denominator in return on ad spend. Must be >= 0.

FactUserEvent

One row per action, session boundaries included. The finest grain in the database and the single busiest table, which is why it carries nothing but keys, a timestamp and a handful of short codes.

Table 9
FactUserEvent — column specification.

Column Name	Data Type	Size	Null / NOT Null	Constraint	PK	Purpose of the column
EventID	INT	4	NOT NULL	PK_UserEvent	Yes	Surrogate key. IDENTITY, because at this volume nothing should be assigning keys by hand.
UserID	INT	4	NOT NULL	FK_Event_User		Who acted. Mandatory; this is how an event reaches a person.
SessionID	INT	4	NOT NULL			Groups events into one sitting. A degenerate dimension: it needs no table of its own, because session length is just the span of the events that carry the same value.
GeoID	INT	4	NOT NULL	FK_Event_Geo		Where the event happened. Compared against the person's registered location to spot an account being used somewhere new.
EventType	VARCHAR	20	NOT NULL	CK_Event_Type		SessionStart, SessionEnd, LevelStart, LevelComplete, LevelFail, ItemUsed or AdWatched. One vocabulary for the whole stream.
EventTS	DATETIME2	8	NOT NULL			When it happened. Indexed, because the date part of this column is what daily and monthly active users are counted from.
LevelNumber	INT	4	NULL	CK_Event_Level		Level in play, 1 to 10. NULL for events that do not belong to a level, such as an ad watch.
Outcome	VARCHAR	10	NULL	CK_Event_Outcome		Win, Loss or Abandon. Only level events carry one. This is what level difficulty is measured from.
DeviceType	VARCHAR	20	NOT NULL	CK_Event_Device		The device this event came from, which is not always the person's usual one.
SourceSystem	VARCHAR	30	NOT NULL	CK_Event_Source		GameClient, MobileSDK or WebClient. Which system emitted the row, so several feeds can land in one table and still be told apart.

FactPayment

One row per payment: store sales, subscription charges and refunds, each tagged with where the transaction came from.

Table 10
FactPayment — column specification.

Column Name	Data Type	Size	Null / NOT Null	Constraint	PK	Purpose of the column
PaymentID	INT	4	NOT NULL	PK_Payment	Yes	Surrogate key for one payment. IDENTITY.
AccountID	INT	4	NOT NULL	FK_Payment_Account		The account charged. Mandatory.
ProductID	INT	4	NULL	FK_Payment_Product		What was bought. NULL on a plain subscription charge, which buys no catalog item.

Column Name	Data Type	Size	Null / NOT Null	Constraint	PK	Purpose of the column
CampaignID	INT	4	NULL	FK_Payment_Campaign		Campaign credited with the sale. NULL when it is not attributable.
GeoID	INT	4	NOT NULL	FK_Payment_Geo		Where the transaction originated. This is the fraud hook: a payment from a country the person did not register in, or one too far from where they were last seen, is the thing worth reviewing.
PaymentTS	DATETIME2	8	NOT NULL			When the money moved. Indexed; drives monthly revenue.
AmountUSD	DECIMAL	8,2	NOT NULL	CK_Payment_Amount		What was actually charged, which can differ from list price after a discount. Must be > 0.
PaymentType	VARCHAR	12	NOT NULL	CK_Payment_Type		OneTime, Subscription or Refund. Separates store sales from recurring revenue inside one table.
PaymentMethod	VARCHAR	20	NOT NULL	CK_Payment_Method		Card, PayPal, AppStore or GooglePlay. Store fees differ by method.
IsFlagged	BIT	1	NOT NULL	DF 0		Set when the transaction trips a review rule. Stored rather than recomputed so a decision made once is not re-litigated.
SourceSystem	VARCHAR	30	NOT NULL	CK_Payment_Source		BillingAPI, AppStore or PlayStore. Which system reported the payment.

FactSubscription

One row per subscription event. Separate from payments because the events that matter most to retention, a downgrade and a cancel, move no money at all and would have nowhere to live otherwise.

Table 11
FactSubscription — column specification.

Column Name	Data Type	Size	Null / NOT Null	Constraint	PK	Purpose of the column
SubEventID	INT	4	NOT NULL	PK_Subscription	Yes	Surrogate key for one subscription event. IDENTITY.
AccountID	INT	4	NOT NULL	FK_Sub_Account		The account whose subscription changed. Mandatory.
PlanID	INT	4	NOT NULL	FK_Sub_Plan		The plan as of this event.
PrevPlanID	INT	4	NULL	FK_Sub_PrevPlan		The plan they moved from. NULL on a first start. A second reference to the same plan table, which is what makes an upgrade and a downgrade distinguishable.
EventType	VARCHAR	12	NOT NULL	CK_Sub_EventType		Start, Renew, Upgrade, Downgrade, Cancel or Expire. The subscription lifecycle, including the events that move no money.
EventDate	DATE	3	NOT NULL			When the change took effect.

Column Name	Data Type	Size	Null / NOT Null	Constraint	PK	Purpose of the column
MRRDeltaUSD	DECIMAL	8,2	NOT NULL			Change in monthly recurring revenue, negative on a downgrade or a cancel. Summed over time, this column is the MRR trend.
TermMonths	INT	4	NULL	CK_Sub_Term		Committed term. NULL on a cancel or an expiry.
CancelReason	VARCHAR	40	NULL			Why they left. Only populated on a cancel, which is why it is optional, and it is the most useful text in the database.
SourceSystem	VARCHAR	30	NOT NULL	CK_Sub_Source		BillingAPI, AppStore or PlayStore. Which system reported it.

DimPlanAudit

The change history of DimPlan. Written only by a trigger, never by hand.

Table 12
DimPlanAudit — column specification.

Column Name	Data Type	Size	Null / NOT Null	Constraint	PK	Purpose of the column
AuditID	INT	4	NOT NULL	PK_PlanAudit	Yes	Surrogate key. IDENTITY, because the trigger cannot supply a key.
PlanID	INT	4	NOT NULL			Which plan changed. Deliberately NOT a foreign key, so a DELETE audit row survives the row it describes.
ChangeType	VARCHAR	6	NOT NULL	CK_Audit_ChangeType		INSERT, UPDATE or DELETE. Which operation fired the trigger.
OldPlanName	VARCHAR	30	NULL			Plan name before the change. NULL on an insert.
NewPlanName	VARCHAR	30	NULL			Plan name after the change. NULL on a delete.
OldPriceUSD	DECIMAL	6,2	NULL			Price before the change. NULL on an insert.
NewPriceUSD	DECIMAL	6,2	NULL			Price after the change. NULL on a delete.
OldIsPaid	BIT	1	NULL			Paid flag before the change. NULL on an insert.
NewIsPaid	BIT	1	NULL			Paid flag after the change. NULL on a delete.
ChangedBy	VARCHAR	128	NOT NULL	DF SUSER_SNAME()		The login that made the change.
ChangedAt	DATETIME2	8	NOT NULL	DF SYSDATETIME()		When the change happened. This is the datetime column the assignment requires on the audit table.